HyperionDev

Control Structures – For Loop

Task

Visit our website

Introduction

In this task, you will be exposed to loop statements to understand how they can be used in reducing lengthy code, preventing coding errors, and paving the way towards code reusability. The looping structure you will learn about in this task is the for loop, which is essentially a different variation of the while loop.

Take note

Python is a high-level language, meaning it is closer to human languages than machine languages. Therefore, it is easier to understand and write. It is also more or less independent of a particular type of computer.

Fortran was the first high-level language. Fortran was invented in 1957 by IBM's John Backus. The name Fortran is derived from "Formula Translation." The language is best suited to numeric computation and scientific computing. Fortran is still used in computationally intensive areas such as numerical weather prediction, finite element analysis, computational fluid dynamics, computational physics, crystallography, and computational chemistry.

For loops

A for loop is similar to a while loop. Either a for loop or a while loop can be used to repeat instructions. However, unlike a while loop, the number of repetitions in a for loop is known ahead of time. A for loop is a counter-controlled loop. It begins with a start value and counts up to an end value. A for loop allows for counter-controlled repetition to be written more compactly and clearly. Therefore, a for loop is easier to read.

In Python, a for loop has the following syntax:

```
for index_variable in sequence:
            statements
```

As you can see, the Python for loop starts with the keyword `for`, followed by a variable that will hold each of the values of the sequence as we move through it. The index variable can tell you what iteration the loop is on.

In each iteration (or repetition) of the for loop the code that is indented is repeated. The Python `range()` function generates a sequence of numbers, which are used to iterate through a for loop. `The range()` function needs two integer values, a start number and a stop number. For the function `range(start index: end index)`, the start index **is included** and the end index is **not included**.

In the for loop below, while the variable `i` (which is an integer) is in the range of 1 to 10 (i.e. either 1, 2, 3 , 4, 5, 6, 7, 8, or 9), the indented code in the body of the loop will execute. The `range(1, 10)` specifies that `i = 1` in the first iteration of the loop, so `1` will be printed in the first iteration of the code example below. Then the code will run again, this time with `i = 2`, and `2` will be printed out, etc., until `i = 10`. At this point, `i` is no longer in the `range (1,10)` (remember that the end index is excluded), so the code will stop executing.

`i` is known as the index variable as it can tell you the iteration or repetition that the loop is on. In each iteration of the for loop, the code indented inside is repeated.

```
for i in range(1, 10):
    print(i)
```

This for loop in the example above prints the numbers one to nine. Again, note the indentation and the colon, just like in the if statement.

You can use an if statement within a for loop!

```
for i in range (1,10):
    if i > 5:
        print(i)
```

The code in the example above will only print the numbers 6, 7, 8, and 9 because numbers less than or equal to five are filtered out.

For a for loop to function properly, the following things must happen:

- **Initialise loop**: The loop needs to use a variable as its counter variable. This variable will tell the computer how many times to execute the loop.
- **Loop test**: The loop test is a boolean expression in Python that evaluates to either True or False. The loop test expression is evaluated before any iteration of the for loop. If the condition is True, then the program control is passed to the loop body; if False, control passes to the first statement after the loop body.
- **Update statement**: Update statements assign new values to the loop control variables. The statement typically uses the increment `i+=1` to update the control variable. An update statement is always executed after the body has been executed. After the update statement has been executed, control passes to the loop test to mark the beginning of the next iteration.

A loop could also contain a break statement. Within a loop body, a break statement causes an immediate exit from the loop to the first statement after the loop body. The break allows for an exit at any intermediate statement in the loop. Have a look at the example below. Copy and paste it and try it out!

```
num_list = [1, 2, 3, 4, 5]
found = False
num = int(input("Input a number from 1 to 10 and find out if it's in our list:"))
if num<1 or num>10:
      print("Number out of range")
else:
      for number in num_list:
                if num == number:
                        found = True
                        break
      print(f'List contains {num}: {found}')
```

Code hack

Using a break statement to exit a loop has some important applications in working with files. Imagine a program which uses a for loop to input data from a file. The number of iterations of the for loop will depend on the amount of data in the file. The task of reading from the file is part of the loop body, which becomes the place where the program discovers that data is exhausted. When the end-of-file condition becomes True, a break statement can be used to exit the loop.

In selecting a loop construct (either while loop or for loop) to read from a file, we recognise that the test for end-of-file occurs within the loop body. The loop statement has the form of an infinite loop: one that runs forever. The assumption is that we do not know how much data is in the file. Versions of the for loop and the while loop permit a programmer to create an infinite loop. In the for loop, each field of the loop is empty. There are no control variables and no loop test. The equivalent while loop uses the constant True as the logical expression.

The syntax of infinite for and while loops looks as follows:

```
for range:
      loop block

while true:
      loop block
```

Nested loops

A nested loop is simply a loop within a loop. Each time the outer loop is executed, the inner loop is executed right from the start. That is, **all the iterations of the inner loop are executed with each iteration of the outer loop.**

The syntax for a nested for loop in another for loop is as follows:

```
for iterating_var_1 in sequence:
      for iterating_var_2 in sequence:
            statements(s)
      statements(s)
```

The syntax for a nested **while loop in another while loop** is as follows:

```
while condition_1:
      while condition_2:
            statement(s)
      statement(s)
```

You can put any type of loop inside of any other kind of loop. The syntax for a nested **for loop in a while loop**, for example, is as follows:

```
for iterating_var in sequence:
      while condition:
            statement(s)
      statements(s)
```

The following program shows the potential of a nested loop, in this case used to output times tables:

```
for x in range(1, 6):
    for y in range(1, 6):
        print(f"{x} * {y} = {x*y}")
    print("")
```

When the above code is executed, it produces the following result:

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15

4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
```

Trace tables

As you start to write more complex code, **trace tables** can be a really useful way of desk-checking your code to make sure the logic of it makes sense. You can do this by creating a table where you fill in the values of the variables for each iteration. This is particularly useful when you are iterating through nested loops. For example, let's look back to the code above and create a trace table:

x	y	x*y
1	1 2 3 4 5	1 2 3 4 5
2	1 2 3 4 5	2 4 6 8 10
3	1 2 3 4 5	3 6 9 12 15
4	1 2 3 4 5	4 8 12 16 20
5	1 2 3 4 5	5 10 15 20 25

As you can see, because of the nature of nested loops, the inner loop iterates five times before the outer loop is iterated again. That means that x will remain the same value while y loops through all its iterations. Then, once the outer loop iterates, the inner loop restarts with another five iterations. This is represented by the trace table, where y cycles from one-five for the same value of x. Practise drawing trace tables for your upcoming tasks to assist you with the logic – they can be very helpful when coding gets tricky!

Take note

Read and run the accompanying **examples files** provided before doing the practical task to become more comfortable with the concepts covered in this task. Feel free to tweak the code there to see what happens – this can be a valuable learning mechanism!

Practical task

Follow these steps:

- Create a new Python file in this folder called **pattern.py**.
- Write code to output the arrow pattern shown below, using an *if-else statement* in combination with a *for loop*
    - You are **allowed** to use more than one for loop. But use only one for loop if you wish to challenge yourself):

```
*
**
***
****
*****
****
***
**
*
```

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
